

Post-Quantum State Channel

Verifiable Custodial Payment System on Bitcoin

Introduction

This document describes a Post-Quantum Custodial Verifiable State Channel designed to use BTC as a method of payment in daily life, preserving user privacy, security, and public verifiability of the entire system.

This is not a 100% trustless system: it is a custodial system where trust is not requested blindly but is demonstrated mathematically. Anyone can verify that the rules have been respected. No one can see who paid what to whom. Trust is not required — it is proven. Mathematically. Publicly. By anyone who wants to verify it.

With the BitVM2 Bridge, introducing a 1-of-N trust model with symmetric bidirectional slashing. The boundary between the off-chain ledger and Bitcoin L1 moves from a mechanism based on trust in a committee of signers to a mechanism where incorrect behavior is automatically punished on-chain by Bitcoin itself.

1. Signature

The signature scheme is ML-DSA-44, approved by NIST (FIPS 204), chosen for its optimal balance between security and overhead. It is based on two problems that are computationally hard even for quantum computers: Module Learning With Errors (MLWE) and Module Short Integer Solution (MSIS). It uses Fiat-Shamir to make the protocol non-interactive and NTT to accelerate internal multiplications.

The signature is used to co-sign checkpoints between the phone (A) and the POS (B), and for identity key certification of devices by the Core.

Sizes

- pk: 1312 bytes (fixed)
- sig: 2420 bytes (fixed)
- sk: 2560 bytes (library-dependent)

Digest to Sign

```
m = H("Sc/v1/sig" || CBOR_canonical(CheckpointCore))
```

The signing process uses Hedged mode (mandatory to mitigate weak RNGs and side-channels). Verification reconstructs the public matrix from the seed in pk, recalculates the challenge via Fiat-Shamir, and verifies consistency.

2. A–B Messages

Phone (A) and POS (B) communicate in CBOR_canonical format to guarantee: same structure = same bytes always. This is essential for co-signing since both must sign identical bytes.

The POS constructs $C_{\{k+1\}}$, both devices sign it independently with ML-DSA-44, and the POS forwards the co-signed checkpoint to the Core via Protobuf over mTLS. The Core never constructs checkpoints — it only verifies and records them.

Checkpoint Structure

```
Checkpoint = CheckpointCore + sig_a + sig_b
CheckpointCore = { balance_a, balance_b, channel_id, epoch_id,
                  fee_bucket (optional), k, network_id, version }
```

Pre-Signing Checks

- $k = \text{last_k} + 1$
- balance_a and $\text{balance_b} \geq 0$
- Value conservation respected
- epoch_id consistent with the time window
- No checkpoint accepted during the freeze window (60–120 seconds)

Device A Verification Requirement (Mandatory Firmware)

Before co-signing $C_{\{k+1\}}$, device A must autonomously verify that the bytes received from the POS correspond to what Alice authorized. This verification is performed automatically by the application — the user does not see it and does not need to do anything manually.

The standard checks — $k = \text{last_k} + 1$, non-negative balances, value conservation, epoch consistency, freeze window — are verified independently by both device A and the Core. The additional check that only device A can perform is:

```
balance_a_received == balance_a_previous - amount_authorized_by_Alice
```

This is the only check not already in the standard list, and it is the one that protects Alice from a tampered POS that constructs a formally valid $C_{\{k+1\}}$ — passing all standard rules — but transferring more value than Alice authorized. If the check fails, device A refuses to sign and notifies Alice with an amount mismatch error. The transaction does not proceed.

3. B-Core Messages

The POS sends a header and body to the Core via Protobuf over an mTLS channel. Serialization is deterministic. The mTLS certificate identifies the sender POS.

Core Verifications

- mTLS authentication and binding with `submitter_id`

- Deduplication: pair (submitter_id, nonce_tx) within ttl_sec — protection against retry storms
- Application idempotency beyond ttl_sec: verifies (channel_id, k, C_k, signatures)
- Header checks: version, network, epoch consistency
- Ledger rules: $k = \text{last_k} + 1$, value conservation, non-negative balances, freeze window

Responses

- ACK_ACCEPTED: commit and state_root update
- ACK_DUPLICATE: duplicate, no update
- NACK_INVALID: invalid, no point in retrying
- NACK_TEMPORARY: freeze window active, retry after

4. Sequence Numbers — Layered Protection

Three-level protection against replay, duplicates, and tampering: network, channel, global.

Network Level

Actors: submitter_id, nonce_tx, ttl_sec. If the pair (submitter_id, nonce_tx) has not been seen within ttl_sec: proceed. If seen with same payload: duplicate without recalculation. If seen with different payload: error (nonce reuse).

Channel Level

Monotonic k: $k = \text{last_k} + 1$. Protects against old states ($k \leq \text{last_k}$), equivocations (same k, different content: first wins), and out-of-order messages.

Global Level

tx_id = stable hash of canonical bytes. Recognizes same content even across different submitter_ids, expired ttl_sec, or different batches. Basis for global audit. The AEAD nonce is separate from nonce_tx and concerns the payment session: each encrypted message has a unique nonce for the session key derived from CTIDH.

5. Ledger / Merkle Tree

Objectives

- Verifiable immutability: every update produces a new public state_root
- Local inclusion proofs: anyone can verify their own channel/balance in the root
- Conservation audit: roots allow verification of entries/exits/balances

Sparse Merkle Tree (SMT)

A key-value map in 2^{256} space. Each leaf has an exact position determined by key and epoch (salt). Default hash $D(i)$ for empty nodes, salted with the epoch to avoid predictable patterns.

Key Derivation and Hash

```
Key = H("keynul" || epoch_id || leaf_type || identifier)
hash_leaf = H("Leafnul" || key || h_val)
h_val = H("Valnul" || canonical_value)
D0 = H("Emptynul" || epoch || 0)
D(i+1) = H("Emptynul" || epoch || D(i) || D(i))
```

Root Calculation

Derive 256-bit key (Big Endian MSB->LSB path, 0=left, 1=right). Start from hash_leaf, ascend toward root. Left: $h_parent = H("Nodenul" || h_leaf || h_sibling)$. Right: $h_parent = H("Nodenul" || h_sibling || h_leaf)$. Repeat for 256 levels.

Leaf Types

The SMT supports multiple leaf types, each identified by a leaf_type field in the key derivation:

- CHANNEL: state of an open or closed channel, identified by channel_id. Contains k, balance_a, balance_b, status.
- CHALLENGE_HISTORY: challenge record for a participant commitment, identified by identity_commitment. Contains number of lost challenges, txid of each lost challenge, current collateral multiplier. Updated by the Core after every lost challenge. Enables on-chain verification that the authorization token issued by the Core corresponds to the real challenge history.
- MERCHANT_CREDIT: accumulated payment credit for a merchant, identified by merchant_id. Contains the total BTC balance owed to the merchant from all closed channels across all their registered terminals. Updated by the Core on every channel closure — regardless of closure type (EOD, user_force_close, RISK) — by adding the final balance_b of the closed channel to the merchant's accumulated credit. Settlement of the accumulated credit occurs at EOD or on merchant request, aggregated in a single batch transaction.
- BATCH_STATE: state of an ongoing peg-out batch, identified by batch_id. Contains for each user in the batch: amount_due (the original amount requested), amount_confirmed_received (what has actually arrived on-chain), missing_delta (the residual amount still owed), and transaction_status (CONFIRMED_CORRECT, CONFIRMED_INCORRECT, UNCONFIRMED). Updated by the Core after every on-chain confirmation related to the batch. Enables external verification that any subsequent batch correctly reconciles previously incorrect or missing payments.

Tombstone

On closure, the leaf is never deleted but marked with a tombstone attesting the closure with final balances. History is always verifiable, even after EOD.

6. Zero Knowledge

Complete transparency of rules. Complete privacy of people. ZK proofs guarantee the system operated correctly without revealing who paid what to whom.

Choice: LaBRADOR with Migration Plan to Ligerio/Brakedown

LaBRADOR (lattice-based, Module-SIS, natively PQ) is the current choice for proof compactness, essential for OP_RETURN anchoring. Conscious choice with known risks: young system (2023) with less scrutiny than alternative systems. Migration plan: Ligerio/Brakedown (hash-based, stronger PQ posture, sublinear proofs with Brakedown).

Statement and Witness

- Public statement (visible to all): epoch, root_in, root_out
- Secret witness (commitment): ordered SMT update list, sibling nodes for tree recalculation, sensitive data (amounts, identities, mappings)

Internally Verified Constraints

- Correctness of SMT update
- $k = \text{last_k} + 1$ for each channel
- Value conservation (no coin creation)
- No negative balances
- Correct epoch
- No repeated IDs

SHAKE256/Poseidon Bridge

For each node touched by an update, the witness contains the plaintext value v , $P(n) = \text{Poseidon}(\text{prefix} || v)$ for the internal ZK circuit hash, and $S(n) = \text{SHAKE256}(\text{prefix} || v)$ for the external hash. The ZK circuit verifies $P(n)$ internally. The external verifier independently computes $\text{SHAKE256}(\text{prefix} || v)$ and compares with $S(n)$. The commitment binds the prover to a single v : a malicious prover would need to break both hash functions simultaneously.

7. Bridge Layer — BitVM2

The bridge layer manages the boundary between the off-chain ledger and Bitcoin L1. This version uses the BitVM2 Bridge, moving from a majority-trust model (t-of-n) to an existential trust model (1-of-N). In the 1-of-N model, a single honest participant is

sufficient to guarantee system security. Incorrect behavior is not merely detectable — it is automatically punished by Bitcoin L1 via on-chain slashing.

Bridge Architecture: BitVM2 with Unified Identity System

The bridge layer is based on the BitVM2 protocol. The system extends BitVM2 with an identity and reputation architecture described below. I am aware that what follows is not complete with regard to the entire Bridge architecture; these are ideas I have worked on in order to bring them into the document to the reader's attention and to receive serious constructive criticism, feedback and/or ideas.

Unified On-Chain Identity Commitment

All participants — operators, challengers, users, watchtowers, and the Core — register a pseudonymous identity commitment on Bitcoin via OP_RETURN at onboarding:

```
commitment = H(identifier || salt || epoch_registration)
```

The commitment is permanently anchored on Bitcoin. It reveals nothing about the participant's real-world identity. It creates a persistent pseudonymous identifier that accumulates reputation on-chain through the participant's history of challenges, slashing events, and setup participations.

The same commitment mechanism applies to all roles. What differs is how punishment is applied when the commitment's holder misbehaves.

Asymmetric Punishment by Role

- Challengers: progressive cumulative collateral. Each lost challenge increases the collateral required for the next challenge: $2^N \times \text{minimum}$, where N is the number of lost challenges for that commitment. A won challenge does not reduce the accumulated requirement. The collateral requirement resets after a prolonged period of clean conduct, defined as a system parameter.
- Operators: immediate total slashing. A false Claim proven via on-chain Disprove results in immediate loss of the full stake. Re-entry requires a higher stake than the previous instance — an escalating entry cost that makes repeated misbehavior progressively more expensive.
- Core: open question — see Section 9.

Permissionless Setup with Objective Admission Requirements

The setup committee for each bridge instance is not a fixed permissioned list. Any participant whose commitment satisfies the following publicly defined and on-chain verifiable conditions may join:

- Slashing history within acceptable bounds: (a) no slashing events in the last CLEAN_WINDOW blocks (system parameter); (b) no more than MAX_LIFETIME_SLASHES slashing events across the entire commitment history (suggested default: 3). A commitment with zero slashing events

always qualifies. A commitment that has been slashed but not within `CLEAN_WINDOW` and below `MAX_LIFETIME_SLASHES` also qualifies — this allows rehabilitation over time without granting permanent exclusion from the system.

- Stake deposit $\geq$ `MIN_STAKE_PER_OPERATOR` (system parameter), deposited individually and non-transferable to other participants. Operators with slashing events in their history must deposit a higher stake on re-entry — the exact multiplier is a system parameter to be defined, deliberately set high enough to make re-entry economically discouraging after each slash. The deterrent effect must increase cumulatively so that after 2-3 slashing events the cost of re-entry significantly exceeds the potential gain from further misbehavior. This obligation persists regardless of whether the last slash falls within `CLEAN_WINDOW` or not: `CLEAN_WINDOW` determines eligibility, not the stake requirement.
- Commitment registered before the instance's admission window closes

The instance defines three parameters that must all be satisfied simultaneously before the setup proceeds: `MIN_OPERATORS` (minimum number of participating operators, guaranteeing the 1-of-N honesty model), `MAX_OPERATORS` (maximum number of operators per instance), and `MIN_STAKE_PER_OPERATOR` (minimum individual stake each operator must deposit). The setup starts when the number of admitted operators is between `MIN_OPERATORS` and `MAX_OPERATORS` or at least equal to the `MIN_OPERATORS` and every operator has individually deposited at least `MIN_STAKE_PER_OPERATOR`. If the admission window closes before `MIN_OPERATORS` is reached, deposited stakes are returned via timelock and a new window opens. When candidates exceed `MAX_OPERATORS`, priority is assigned by cleanest slashing history first, then highest stake, then earliest registration.

Individual stakes are non-transferable and non-redistributable. An operator who deposits more than `MIN_STAKE_PER_OPERATOR` — whether voluntarily or because slashing history requires a higher deposit — does not lower the requirement for other participants. Each operator's stake requirement is calculated independently. Excess stake above the minimum remains the operator's own collateral at risk.

The rules are objective and public. No committee decides who is admitted — the on-chain record of each commitment decides. An operator with a clean history and sufficient stake cannot be excluded by other participants' preference.

Honest Limits of Permissionless Setup

Bitcoin cannot force a participant to sign the transaction graph. The setup requires active cooperation from all admitted participants. A single participant who refuses to co-sign blocks the instance, regardless of whether their reason is legitimate or arbitrary.

Setup inactivity is an open problem. Registering on-chain when a participant was admitted to a setup but did not co-sign the transaction graph requires solving two sub-problems: separating the commitment to participate from the act of signing, and

defining consequences for failing to sign after committing. The punishment mechanism for non-participation is left as an open question — see Section 9.

Challenge Authorization via Core Token

When a participant wants to challenge an operator, they request a challenge authorization token from the Core. The Core:

- Reads the CHALLENGE_HISTORY leaf of the participant's commitment from the SMT
- Calculates the required collateral: $2^N \times \text{minimum}$, where N = number of lost challenges for that commitment
- Issues a time-limited single-use authorization token signed with ML-DSA-44, embedding the required collateral amount and the commitment reference

The challenge transaction published on Bitcoin includes the authorization token. The pre-signed transaction graph verifies that the deposited collateral matches the amount in the token and that the token is signed by the Core's registered key.

If the Core issues a manipulated token anyone can prove the manipulation by presenting the SMT inclusion proof for the CHALLENGE_HISTORY leaf against the current state_root anchored on Bitcoin. The ZK proof guarantees that the state_root was constructed honestly, making the manipulation provable to any external verifier without the Core's cooperation.

Open question: the slashing mechanism for a Core that issues manipulated tokens is not yet defined. This is one of the primary points for technical discussion — see Section 9.

Operator On-Chain History and Committee Transparency

An operator's commitment accumulates a public on-chain history: slashing events and challenge outcomes. Any participant evaluating whether to co-sign a setup with a given operator can read this history directly from Bitcoin without trusting any intermediary. Setup participations are not included as a reliable history signal because the mechanism to register and punish non-participation in a setup is an open design question — see Section 9.

This transforms the trust model from 'trust the committee's judgment about who is reliable' to 'verify the commitment's on-chain record and apply the objective admission rules'. Reputation is earned on-chain and cannot be faked or selectively disclosed.

Bridge Phases

Setup

The signer committee pre-signs the entire transaction graph. Each honest signer deletes their key after pre-signing. The bridge is operational but empty. For the initial

deployment, the minimum committee consists of the Core and at least one independent external operator — one honest participant is sufficient.

Peg-In (User Deposit)

Alice deposits BTC via a PegIn transaction. The Core detects the deposit and credits BTC in the offline ledger via the PENDING/COMPLETED atomic pattern. Alice receives tokens in the ledger equivalent to the deposited BTC.

Peg-Out (User Withdrawal)

Bob burns tokens in the ledger and creates an incomplete PegOut transaction with SIGHASH_SINGLE|ANYONECANPAY. Operators compete to complete the transaction by adding their own input and publishing on Bitcoin. The winning operator advances BTC from their own funds and pays Bob immediately.

Claim and Challenge

The operator submits a Claim Transaction to recover the advanced BTC from the bridge. If uncontested within period Δ_B , the operator publishes PayoutOptimistic and recovers funds. If challenged, the operator must publish the Assert Transaction with all intermediate SNARG computation states. If a state is false, the challenger publishes the Disprove Transaction executing that specific sub-program on-chain. Bitcoin itself verifies the error and slashes the operator's stake.

EOD Settlement: Asymmetric Design

Settlement at end of day is asymmetric by design:

- Merchants: automatic EOD settlement on-chain. The merchant has delivered goods or services and has a legitimate interest in receiving BTC on L1 at day's end. Settlement is aggregated in a single batch per merchant to minimize on-chain transactions and operator liquidity requirements. Merchants who prefer to maintain liquidity in the system can opt out.
- Users: no automatic settlement. Funds remain in the bridge until the user explicitly requests a peg-out. This avoids forcing users to re-deposit every day.

Batch Peg-Out with Core Reconciliation

The fixed instance amount in BitVM2 bridges is incompatible with retail use cases where users hold small balances. To resolve this without compromising verifiability, peg-out requests are aggregated into batches that match the instance amount.

Normal flow: the Core accumulates peg-out requests in a waiting queue. When the total reaches the instance threshold, the batch is ready. An operator advances the funds by constructing a single Bitcoin transaction with one output per user in the batch (each with their specific amount) plus the Claim toward the bridge. Distribution and Claim are in the same transaction, in the same block — fully atomic.

Fraud detection: watchtowers monitor every block in real time. When a Claim appears, they automatically verify that each output corresponds to the batch

mapping registered in the Core via `state_root`. If discrepancies are detected within the 48-hour challenge window, the watchtower publishes the challenge.

Reconciliation after fraud: depending on what is already confirmed on-chain when the challenge is published, the Core classifies each user in the batch into three states: `CONFIRMED_CORRECT` (user received the correct amount, entry closed), `CONFIRMED_INCORRECT` (user received less than owed, the Core records the missing delta and includes it in the next batch), `UNCONFIRMED` (transaction was in mempool and never confirmed, the user re-enters the next batch for the full original amount). The Core tracks all of this in the `BATCH_STATE` SMT leaf, with `state_root` updated after every on-chain confirmation.

Fourth case of legitimate challenge: the Claim includes distributions with incorrect amounts or missing users compared to the batch mapping registered in the Core. Directly verifiable on-chain by comparing the transaction outputs with the public `state_root`.

State Root Anchoring on Bitcoin

Every ledger update produces a new `state_root`. This is anchored on Bitcoin via `OP_RETURN`, creating an immutable and temporally ordered chain of states.

`OP_RETURN` content: `H(state_root || epoch || block_height_ref) - 32 bytes`

Current anchoring frequency: once per day (EOD). Future roadmap: every 10 minutes synchronized with Bitcoin blocks via pre-signed transactions.

Watchtower and Incentives

Watchtowers are automated software that monitor operator Claims 24/7 and contest false ones. Incentive model: base fee from every ledger transaction contributes to a common fund remunerating watchtowers even when no fraud occurs. Additional reward if a watchtower finds and proves fraud. Watchtowers deposit stake to challenge — if founded they win the reward, if unfounded they lose the stake.

Crowdfunding allows watchtowers with limited capital to collaborate on covering high collateral requirements, lowering entry barriers.

Terminal Identifier

`channel_id = f(card_id, pos_id, epoch)`, where `pos_id` is the Terminal ID (TID) — a unique identifier assigned to each individual POS device. This mirrors the standard payment industry convention and ensures that multiple terminals of the same merchant do not share a channel, eliminating concurrency conflicts.

Bridge Setup Verification

It is not possible to cryptographically prove that a private key was deleted after setup. Practical guarantees: open-source operator software, public pre-signed transactions verifiable before any deposit, at least one independent external operator

certifying key deletion, Bitcoin's automatic invalidation of any transaction violating the pre-signed graph.

8. State Machine

Participants

- User A (card / phone)
- Merchant B (POS, onboarded via one-time ML-DSA-44 signed binding $H(\text{'registration' || pos_id || merchant_id || epoch})$, verified by the Core before any channel can be opened)
- Core (ledger + ZK + BitVM2 bridge + Bitcoin L1 connection)
- Operators (advance funds for peg-out)
- Watchtowers (monitor operator Claims)

Channel Identifier

$\text{channel_id} = f(\text{card_id}, \text{pos_id}, \text{epoch})$. For each (card_id, pos_id) pair, at most one open channel exists. The channel is opened at the first payment of the day and closed on user request, automatically at EOD, or on a risk event.

Channel States

- NON_EXISTENT: channel not created or closed with tombstone
- ACTIVE: open channel, no in-flight transaction, confirmed checkpoint C_k ($k \geq 0$)
- PENDING: $C_{\{k+1\}}$ signed by A and B but not yet confirmed by Core. No other transaction accepted
- CLOSING: closure initiated. No new C_k . Reason: USER, EOD, or RISK
- CLOSED: closure completed. Leaf marked tombstone. No operations permitted

State Transitions

Open (open_channel): NON_EXISTENT → ACTIVE. NFC contact, Core verifies no open channel for the pair, verifies bridge funds, A and B sign C_0 . Core records C_0 , $\text{last_k} = 0$, state ACTIVE.

Payment (checkpoint): ACTIVE → PENDING → ACTIVE. POS constructs $C_{\{k+1\}}$, both sign, channel enters PENDING. Core verifies signatures, $k = \text{last_k} + 1$, rules. If ok: updates ledger, returns ACTIVE. If rejected: returns ACTIVE with last_k unchanged.

User closure (user_force_close): ACTIVE/PENDING → CLOSING → CLOSED. The user submits a request with the last checkpoint. Core selects maximum valid k . On closure, the Core updates the MERCHANT_CREDIT leaf of the corresponding

merchant_id in the SMT, adding balance_b to the merchant's accumulated credit. Settlement behavior for the user follows the configured Settlement Preference

EOD closure (auto_close): ACTIVE/PENDING -> CLOSING -> CLOSED. Freeze window: no new channels or checkpoints. For each unclosed channel: if PENDING, Core decides to confirm or revert to last valid k. Passes to CLOSING, calculates final balances. On closure, the Core updates the MERCHANT_CREDIT leaf of the corresponding merchant_id in the SMT, adding balance_b to the merchant's accumulated credit. At EOD, the Core produces one aggregated settlement batch per merchant_id, paying the total accumulated MERCHANT_CREDIT in a single transaction with ZK proof and OP_RETURN anchoring. For users, settlement behavior follows the configured Settlement Preference.

Risk closure (RISK): ACTIVE/PENDING -> CLOSING -> CLOSED. Lost card or POS reported. Core disables compromised identity, closes all associated channels with maximum valid k. On closure, the Core updates the MERCHANT_CREDIT leaf of the corresponding merchant_id in the SMT, adding balance_b to the merchant's accumulated credit. Merchant credit is included in the next EOD batch. User settlement follows the configured Settlement Preference.

Settlement Preference (User-Configurable)

Users configure how the Core behaves at channel closure regarding on-chain settlement. The available options differ by closure type. Note: the Core cannot execute a peg-out on behalf of the user, peg-out requires the user's own signature on the PegOut transaction.

EOD closure

- Always settle on-chain: at EOD the Core notifies the user's device, which automatically signs and broadcasts the PegOut transaction. For users who always want BTC on L1 at end of day.
- Notify and wait: the Core sends a notification at EOD and waits for explicit user confirmation within a timeout window. If the user confirms: the device signs and broadcasts the PegOut transaction. If the user does not respond or declines: funds remain in the ledger available the next day. This is the default mode.
- Manual only: no automatic notifications at EOD. Funds always remain in the ledger until the user explicitly initiates a peg-out from their device.

User force close

- Always settle on-chain: at channel closure the Core notifies the user's device, which automatically signs and broadcasts the PegOut transaction.
- Manual only: the channel closes but funds remain in the ledger until the user explicitly initiates a peg-out.

Risk closure

- Risk closure: the Core closes the channel, disables the compromised identity in the ledger, and updates the MERCHANT_CREDIT leaf. However, the Core

cannot execute a peg-out on behalf of the user — peg-out requires the user's ML-DSA-44 private key signature directly on the bridge. If the device is lost or compromised, the attacker who holds the private key can construct a PegOut transaction toward any Bitcoin address and broadcast it directly to the bridge, bypassing the Core entirely. The Core's revocation of the device certificate prevents new checkpoint signatures in the ledger but does not block a PegOut transaction already signed with the compromised key. Settlement behavior in risk closure is therefore an open question — see Section 9

Default behavior on silence: if the user does not respond to a notification, the Core never executes on-chain settlement automatically. Silence always means funds remain in the ledger. The Core must never perform irreversible on-chain operations without explicit user consent, except in escalated risk events where fund protection requires immediate action.

9. Open Vulnerabilities and Unresolved Questions

This section documents known open problems. Intellectual honesty about unresolved issues is more useful than a false picture of completeness.

1. Griefing Attack on Small Amounts (Bridge)

If the cost of challenging an operator's Claim exceeds the contested amount, no rational challenger will intervene. A corrupt operator could steal small amounts below the economic challenge threshold.

2. Tampered POS and Firmware Integrity

A tampered POS can construct a $C_{\{k+1\}}$ that is formally valid — passing all standard checks (k, non-negative balances, value conservation, epoch, freeze window) — but transfers more value than Alice authorized. This attack is not detectable by the Core because all ledger rules are satisfied.

The only protection against this attack is device A's firmware verification: $\text{balance_a_received} == \text{balance_a_previous} - \text{amount_authorized}$. This check is not part of the standard pre-signing rules and must be explicitly implemented in the application firmware of Alice's device.

If the firmware is absent or compromised, this specific attack vector remains open. Firmware certification is therefore a system security requirement. The exact certification mechanism — hardware attestation, reproducible builds, open-source verification — is an open implementation question.

3. Operator Identity Reuse After Slashing

A slashed operator can generate a new key and re-enter as a new operator with a clean stake. Bitcoin has no native persistent identity registry. The cumulative penalty proposal requires an on-chain identity anchoring mechanism across instances — the precise technical mechanism is open.

4. Proof of Key Deletion After Setup

It is not possible to cryptographically prove key deletion after BitVM2 setup. Practical guarantees: open-source software, public setup logs, at least one independent external operator, Bitcoin's automatic invalidation of off-graph transactions. The 1-of-N model means only one honest signer is sufficient.

5. Risk Closure and User Fund Protection

In a risk closure event — lost device or compromised account — the Core closes the channel and disables the identity in the ledger. However, the peg-out requires the user's ML-DSA-44 private key signature directly on the bridge. The Core cannot perform this action on behalf of the user.

If the device is lost, the user cannot sign the peg-out and the funds remain locked in the bridge with no accessible exit path. If the device is compromised, an attacker holding the private key can construct a valid PegOut transaction toward any Bitcoin address and broadcast it directly to the bridge, bypassing the Core entirely. The Core's ability to revoke the device certificate in the ledger does not extend to the bridge — the bridge validates signatures, not Core-issued certificates.

No solution has been identified for this problem. It is presented here as an open question.

6. Core Failure Recovery

The current design does not specify what happens to open channels if the Core goes offline and restarts with lost in-memory state. A robust crash recovery protocol is required before production deployment.

7. Freeze Window and Pending Transactions

If a transaction is in PENDING state when the EOD freeze window begins, it can be held for the next batch or forwarded to a Core instance in a different timezone not currently in its EOD window. Open questions: in which batch does the forwarded transaction settle? What is the cross-Core state synchronization mechanism? These are left open for technical discussion.

8. Setup Inactivity Registration (Open Question)

Registering on-chain when an admitted participant fails to co-sign the transaction graph during a setup requires solving two sub-problems: (a) separating the commitment to participate from the act of signing, so there is a verifiable on-chain record of who promised to participate; and (b) defining what consequences follow from failing to sign after committing.

9. Core Slashing Mechanism (Priority Open Question)

The Core plays a critical role in the challenge authorization mechanism: it reads the CHALLENGE_HISTORY leaf from the SMT and issues tokens that determine the collateral required for each challenge. A Core that issues manipulated tokens, with incorrect collateral amounts, can selectively favor or censor challengers.

While token manipulation is detectable and provable by any external verifier via SMT inclusion proofs and ZK proofs, the current design has no automatic on-chain mechanism to slash the Core's stake in response to proven manipulation. The Core is subject to reputational damage and loss of user trust, but not to automatic economic punishment equivalent to the operator slashing mechanism.

Defining an efficient, trust-minimized Core slashing mechanism is one of the primary open problems of this design.